PYTHON DAY 9

Errors, Exceptions & Classes

Practical + Research-Oriented Learning

Learn → Predict → Run → Break → Debug → Build → Research

Part A — Chapter 8: Errors and Exceptions

Part B — Chapter 9: Classes

A hands-on study guide with examples, debugging labs, mini projects, knowledge checks, and a completion checklist.

Today's Goals

By the end of this guide, you should be able to distinguish syntax errors from exceptions, handle expected failures, raise meaningful errors, clean up resources reliably, and design your own exception types. You will then connect these ideas to object-oriented programming: classes, instances, methods, inheritance, namespaces, iterators, and generators.

> **Rule of thumb:** Use the same learning loop throughout: predict the result first, run the example, change one thing, explain what changed, and record the lesson.

Table of Contents

8. Errors and Exceptions

8.1 Syntax Errors

8.2 Exceptions

8.3 Handling Exceptions

8.4 Raising Exceptions

8.5 Exception Chaining

8.6 User-defined Exceptions

8.7 Defining Clean-up Actions

8.8 Predefined Clean-up Actions

8.9 Raising and Handling Multiple Unrelated Exceptions

8.10 Enriching Exceptions with Notes

9. Classes

9.1 A Word About Names and Objects

9.2 Python Scopes and Namespaces

9.2.1 Scopes and Namespaces Example

9.3 A First Look at Classes

9.3.1 Class Definition Syntax

9.3.2 Class Objects

9.3.3 Instance Objects

9.3.4 Method Objects

9.3.5 Class and Instance Variables

9.4 Random Remarks

9.5 Inheritance

9.5.1 Multiple Inheritance

9.6 Private Variables

9.7 Odds and Ends

9.8 Iterators

9.9 Generators

9.10 Generator Expressions

Practical projects, debugging labs, knowledge checks, and checklist

8. Errors and Exceptions

Errors are not all the same. Some stop Python from understanding your program; others happen while the program is running. Robust programs anticipate likely failures, handle only the cases they can recover from, and allow unexpected problems to remain visible.

8.1. Syntax Errors

A syntax error means Python cannot parse the code as valid Python. The program does not reach normal execution of the invalid statement.

```
if score >= 50
    print("Pass")
```

The condition is missing a colon. Python reports a SyntaxError and points near the location where parsing failed. The highlighted position is a clue, not always the exact cause.

Practice

• Remove a closing parenthesis from a print call and observe the message.

• Mis-indent a line inside an if block.

• Put a string literal between mismatched quotation marks.

• Fix each error and explain the grammar rule that was broken.

8.2. Exceptions

An exception occurs during execution. The code may be syntactically valid, but an operation cannot be completed.

```
print(10 / 0)            # ZeroDivisionError
print(user_name)         # NameError if not defined
int("twelve")            # ValueError
items = [1, 2]
print(items[5])          # IndexError
```

Exception	Typical cause
ZeroDivisionError	Dividing a number by zero
NameError	Using a name that is not defined in the current scope
TypeError	An operation receives an unsuitable type
ValueError	The type is acceptable, but the value is unsuitable
IndexError	A sequence index is outside its valid range
KeyError	A dictionary key is absent
FileNotFoundError	A requested file does not exist at the given path
AttributeError	An object does not provide the requested attribute

Rule of thumb: Do not memorize names alone. Learn to read the traceback from the final exception line upward, then identify the operation that failed.

8.3. Handling Exceptions

Use try for the operation that may fail and except for the particular failure you can handle. The else block runs only when the try block completes without an exception. The finally block runs whether the operation succeeds or fails.

```
raw_age = input("Age: ")

try:
    age = int(raw_age)
```

```
except ValueError:
    print("Enter a whole number.")
else:
    print("Next year you will be", age + 1)
finally:
    print("Age check finished.")
```

Important design choices

• Keep the try block small so you know which operation may fail.

• Catch specific exceptions instead of using a bare except.

• Do not silently ignore a failure; report it, recover, or re-raise it.

• Use else for success-only work, not as a replacement for try.

• Use finally for cleanup that must happen regardless of success or failure.

8.4. Raising Exceptions

raise signals that the current operation cannot continue under the supplied conditions. It can raise a built-in exception or an exception object.

```
def calculate_discount(price, rate):
    if price < 0:
        raise ValueError("price cannot be negative")
    if not 0 <= rate <= 1:
        raise ValueError("rate must be between 0 and 1")
    return price * (1 - rate)

print(calculate_discount(1000, 0.10))
```

Use raise to enforce a contract: valid input is accepted; invalid input is rejected with an informative exception. A caller can decide whether to handle it.

8.5. Exception Chaining

When one exception occurs while handling another, Python can preserve both causes. This is useful when translating a low-level failure into a domain-specific message.

```
def read_quantity(text):
    try:
        return int(text)
    except ValueError as exc:
        raise ValueError("quantity must be a whole number") from exc
```

The expression **from exc** explicitly identifies the original cause. Use **raise NewError(...) from None** only when intentionally suppressing display of the previous context; do not use it to hide problems casually.

8.6. User-defined Exceptions

Create a custom exception when a domain failure deserves a clear, reusable name. Custom exceptions usually inherit from Exception and are named with an Error suffix.

```
class ClaimError(Exception):
    """Base exception for expense-claim problems."""

class ReceiptRequiredError(ClaimError):
    pass

def submit_claim(amount, has_receipt):
    if amount <= 0:
        raise ClaimError("claim amount must be positive")
    if amount > 1000 and not has_receipt:
        raise ReceiptRequiredError("a receipt is required above 1000")
```

A shared base class lets callers handle all claim-related failures together, while a specific subclass lets them respond differently to a missing receipt.

8.7. Defining Clean-up Actions

A finally block is a way to run cleanup logic after a try statement. Cleanup may include closing a resource, releasing a lock, or restoring temporary state.

```
resource = None
try:
    resource = open("notes.txt", "r", encoding="utf-8")
    contents = resource.read()
finally:
    if resource is not None:
        resource.close()
```

The example demonstrates the idea, but Python's with statement is usually clearer for resources that support the context-manager protocol.

8.8. Predefined Clean-up Actions

The with statement uses a context manager to manage setup and cleanup. For files, leaving the with block closes the file even if an exception occurs inside the block.

```
with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()

# The file is closed after the block.
```

Rule of thumb: Prefer with for files and other context-managed resources. It makes the resource lifetime explicit and reduces cleanup mistakes.

8.9. Raising and Handling Multiple Unrelated Exceptions

A single try statement can have several except clauses. Each clause should describe a failure the program can meaningfully handle. Python selects the first matching handler.

```
try:
    count = int(input("Count: "))
    result = 100 / count
except ValueError:
    print("Count must be a whole number.")
except ZeroDivisionError:
    print("Count cannot be zero.")
else:
    print("Result:", result)
```

When different exception types need the same response, a tuple can group them: **except (TypeError, ValueError):**. Avoid grouping exceptions if the correct recovery action differs.

8.10. Enriching Exceptions with Notes

On Python 3.11 and later, BaseException.add_note() can attach extra context to an exception without replacing its type or primary message.

```
try:
    raise ValueError("invalid row")
except ValueError as exc:
    exc.add_note("while importing employee record 42")
    raise
```

Notes are useful for adding operational context such as a record ID, file name, or processing stage. Avoid adding secrets or sensitive personal data to error messages.

Chapter 8 — Debugging Lab

For each scenario, predict the exception, explain why it happens, and decide whether to catch, raise, or allow it to propagate.

1. A user enters 'twenty' where an integer is required.

2. A claim amount is negative.

3. A requested employee ID is absent from a dictionary.

4. A report file is missing.

5. A file opens successfully but reading it fails.

6. A conversion error is caught and re-raised with a more useful message.

Mini Project — Defensive Expense Claim Validator

Build a small validator for an expense claim. Do not start with a full application; implement and test one rule at a time.

• Input: employee ID, category, amount, and receipt status.

• Reject blank employee IDs and non-positive amounts.

• Reject unsupported categories with a meaningful error.

• Require a receipt above a chosen threshold.

• Create a custom ClaimError base class and at least one subclass.

• Use a caller-level try/except to display user-friendly errors.

• Add context notes when a claim fails during batch processing (Python 3.11+).

Suggested test paths

Input / condition	Expected behavior
Valid ID, allowed category, positive amount	Accept claim
Blank ID	Reject with a clear validation error
Amount is text	Report numeric conversion failure
Amount is zero or negative	Raise a validation error
Unsupported category	Report category error
Large claim without receipt	Raise receipt-required error
Unexpected internal error	Do not silently suppress it

9. Classes

A class groups data and behavior into a type of object. Classes help model concepts such as students, employees, library books, expense claims, and appointments. This chapter connects Python's object model to practical program design.

9.1. A Word About Names and Objects

Names refer to objects. Assignment binds a name to an object; it does not necessarily create a new independent copy. Several names can refer to the same mutable object.

```
first = [10, 20]
second = first
second.append(30)
print(first)   # [10, 20, 30]
```

The list is one object referenced by both names. This matters when passing lists or dictionaries to functions and when storing mutable class attributes.

9.2. Python Scopes and Namespaces

A namespace maps names to objects. A scope is the region in which a name can be looked up directly. Python code commonly uses local, enclosing, global, and built-in namespaces.

Scope / namespace	Meaning
Local	Names created inside the current function
Enclosing	Names in an enclosing function, relevant to nested functions
Global	Names defined at module level
Built-in	Names supplied by Python, such as len and print

Python generally resolves names using the LEGB lookup order: Local → Enclosing → Global → Built-in. The global statement allows rebinding a module-level name; nonlocal allows rebinding a name in an enclosing function.

9.2.1. Scopes and Namespaces Example

```
message = "module"

def outer():
    message = "outer"

    def inner():
        message = "inner"
        print(message)

    inner()
    print(message)

outer()
print(message)
```

Predict the three printed lines before running. Each function call has its own local namespace; the local name in inner does not overwrite the outer or module-level binding.

9.3. A First Look at Classes

A class defines attributes and behavior shared by instances. The class statement creates a class object and executes the class body in a new namespace. Methods are functions defined in the class body.

9.3.1. Class Definition Syntax

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

    def introduce(self):
        return f"I am {self.name} ({self.student_id})."
```

__init__ is an initializer called during normal instance creation. The parameter self refers to the instance being operated on; it is a conventional parameter name, not a Python keyword.

9.3.2. Class Objects

A class itself is an object. It can be assigned to a name, passed to a function, and used to create instances. Class attributes and methods can be accessed through the class where appropriate.

```
class Student:
    school = "Vidya Bharathi"

print(Student.school)
```

9.3.3. Instance Objects

An instance is a particular object created from a class. Each instance can hold its own state, typically stored in attributes assigned through self.

```
student_a = Student()
```

The preceding call is only valid if Student has an initializer that accepts no required arguments. Compare that with the Student class defined above, whose constructor requires student_id and name.

9.3.4. Method Objects

Accessing a function defined on a class through an instance produces a bound method: Python supplies the instance as the first argument automatically.

```
class Counter:
    def show(self):
        return "counter"

counter = Counter()
print(counter.show())
```

Calling Counter.show(counter) is conceptually similar to calling counter.show() because the instance is passed explicitly in the first form.

9.3.5. Class and Instance Variables

Class variables are shared through the class unless shadowed by an instance attribute. Instance variables belong to individual instances.

```
class Employee:
    company = "Example Ltd"

    def __init__(self, name):
        self.name = name

first = Employee("Asha")
second = Employee("Ravi")
first.name = "Asha R."
```

Attribute	Typical purpose	Watch out for
Class variable	Shared configuration or class-wide metadata	A mutable list/dict may be shared unintentionally
Instance variable	State specific to one object	Each instance should get its own mutable state

Rule of thumb: Do not use a mutable class variable for per-instance data. Initialize lists and dictionaries inside __init__ when each object needs its own collection.

9.4. Random Remarks

• Attributes can be added to an instance dynamically, but unrestricted attributes can make objects harder to reason about.

• Methods can call other methods using self.method_name(...).

• Objects can contain other objects; composition is often simpler than deep inheritance.

• Use descriptive names and keep each class responsible for a coherent concept.

• Prefer explicit state and clear invariants over unexplained global variables.

9.5. Inheritance

Inheritance creates a new class based on an existing class. A child class can reuse or specialize parent behavior. Use it when there is a genuine 'is-a' relationship, not merely to avoid writing a few lines twice.

```
class Claim:
    def __init__(self, employee_id, amount):
        self.employee_id = employee_id
        self.amount = amount

    def summary(self):
        return f"{self.employee_id}: {self.amount}"

class TravelClaim(Claim):
    def __init__(self, employee_id, amount, destination):
        super().__init__(employee_id, amount)
        self.destination = destination
```

super() delegates to the next class in the method resolution order (MRO). In single inheritance, it is commonly used to initialize the parent part of an object without hard-coding the parent class name.

Overriding a method

```
class Claim:
    def approval_required(self):
        return self.amount >= 1000

class TravelClaim(Claim):
    def approval_required(self):
        return self.amount >= 500
```

The child implementation overrides the inherited method. Keep behavior consistent with the parent class contract so callers can work with either type predictably.

9.5.1. Multiple Inheritance

Multiple inheritance allows a class to inherit from more than one base class. Python uses the method resolution order to decide where attributes and methods are found.

```
class Printable:
    def describe(self):
        return "printable"

class Exportable:
    def export(self):
        return "exported"

class Report(Printable, Exportable):
    pass

report = Report()
```

Multiple inheritance is useful for carefully designed mixins, but complicated hierarchies can make behavior difficult to trace. Inspect the MRO when method lookup is surprising.

9.6. Private Variables

Python does not provide strict private instance variables in the same way some languages do. A single leading underscore, such as _balance, communicates 'internal use by convention'. A double leading underscore triggers name mangling, which helps avoid accidental clashes in subclasses.

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance
```

Name mangling changes the internal attribute name; it is not a security boundary. Use properties when you need to validate assignment or expose a controlled public interface.

9.7. Odds and Ends

• An object can be compared, printed, or used in containers according to the methods its class provides.

• __str__ can provide a readable display string; __repr__ should ideally be useful for debugging.

• __eq__ customizes equality comparisons; define it only when equality has a clear meaning.

• Composition means one object holds or uses another object. It is often a strong alternative to inheritance.

• Type hints can document intended values but do not automatically enforce runtime types.

9.8. Iterators

An iterable can produce an iterator. An iterator remembers its current position and returns one item at a time. The iter() function obtains an iterator; next() requests its next value. When no values remain, the iterator raises StopIteration.

```
numbers = [10, 20, 30]
iterator = iter(numbers)

print(next(iterator))
print(next(iterator))
print(next(iterator))
```

A for loop performs this iteration protocol for you and handles StopIteration at the right boundary. Iterators are useful for processing streams without requiring all values to be stored at once.

9.9. Generators

A generator function uses yield to produce values incrementally. Calling the function returns a generator object; the body runs as values are requested. The function pauses at each yield and resumes on the next request.

```
def count_up_to(limit):
    number = 1
    while number <= limit:
        yield number
        number += 1

for value in count_up_to(3):
    print(value)
```

Generators can reduce memory use for large or streaming data. They are usually single-pass: after exhaustion, requesting more values does not restart the generator.

9.10. Generator Expressions

A generator expression resembles a comprehension but uses parentheses. It creates values lazily, which can be useful when calculating a result from a large sequence.

```
squares = (number * number for number in range(1, 6))
total = sum(squares)
print(total)
```

Construct	Behavior
List comprehension	Builds a list immediately
Generator expression	Produces values lazily as requested
Generator function	Uses yield to define a sequence of produced values

Rule of thumb: Choose a list when you need to revisit or index the values. Choose a generator when you can process each value once and want to avoid storing the entire result.

Practical Lab — Employee Claims

Build a small object-oriented claim model that combines the two chapters. Keep it intentionally small; focus on correct responsibilities and failure handling.

Suggested design

Component	Responsibility
Claim	Store employee ID and amount; provide a summary
TravelClaim	Extend Claim with a destination or travel detail
ClaimError	Base exception for invalid claim operations
ReceiptRequiredError	Specific error when a receipt is missing
Validator function or method	Check amount, category, and receipt requirements
CLI layer	Collect input and show friendly messages

Implementation milestones

1. Write down valid claim rules and invalid cases before designing classes.
2. Define a custom exception hierarchy with one base and one specific exception.
3. Create a Claim class with a clear initializer and a summary method.
4. Create a child class only if it adds meaningful travel-specific behavior.
5. Validate input and raise meaningful exceptions at the boundary.
6. Catch expected errors in the CLI layer; do not put broad exception handling around every method.
7. Add tests for valid input, invalid amounts, missing receipts, unknown categories, and missing IDs.
8. Review class variables to ensure claim-specific data is stored on each instance.

Debugging Lab — Find the Design Fault

Q1. A function prints a computed result, but the caller needs to use it in another calculation. What should the function return?

Q2. Two Employee instances unexpectedly share the same list of claims. Where should that list be initialized?

Q3. A method is called on an instance but Python reports a missing argument. What should you inspect about the method definition and call?

Q4. A child class duplicates the parent's initialization. When might super() be appropriate?

Q5. A file must close even when parsing fails. Which construct is normally the clearest choice?

Q6. A generator produces the expected values once, then produces nothing on a second pass. Why?

Q7. An exception handler catches every possible exception and prints 'failed'. What debugging information might be lost?

Research Challenge

Use the official Python documentation to research each question. Write a short answer in your own words and include a tiny experiment that supports it.

• How does the exception hierarchy help callers catch broad or specific failures?

• What is the difference between a namespace and a scope?

• How does the method resolution order work in multiple inheritance?

• What is the difference between an iterable and an iterator?

• How does yield differ from return?

• When would you choose composition over inheritance?

Knowledge Check — Chapter 8

Q1. What is the difference between a syntax error and an exception?

Q2. Why is catching a specific exception usually better than a bare except?

Q3. What is the purpose of else in a try statement?

Q4. When is finally useful?

Q5. What does raise do?

Q6. Why might you chain one exception from another?

Q7. When is a custom exception helpful?

Q8. What does a with statement do for a file?

Q9. What does add_note() provide, and which Python version introduced it?

Knowledge Check — Chapter 9

Q1. What is the difference between a name and an object?

Q2. Describe local, enclosing, global, and built-in scopes.

Q3. What are a class and an instance?

Q4. What is self used for?

Q5. How do class variables differ from instance variables?

Q6. What does super() help with?

Q7. What is method overriding?

Q8. What is name mangling, and why is it not true security?

Q9. What is an iterator, and what signals exhaustion?

Q10. How does a generator function differ from a generator expression?

Completion Checklist

■ I can distinguish syntax errors from runtime exceptions.

■ I can handle expected errors with specific except clauses.

■ I can raise exceptions and define a custom exception class.

■ I understand exception chaining and exception notes.

■ I can explain try/except/else/finally and the with statement.

■ I can explain names, objects, namespaces, and scopes.

■ I can define a class with instance attributes and methods.

■ I can distinguish class variables from instance variables.

■ I can explain inheritance, overriding, super(), and multiple inheritance.

■ I understand underscore conventions and name mangling.

■ I can explain iterables, iterators, generators, and generator expressions.

■ I completed the Employee Claims practical lab and tested invalid paths.

Recommended Study Schedule

Activity	Time
Read and test Chapter 8 examples	60–75 min
Exception debugging lab + claim validator	45–60 min
Read and test Chapter 9 examples	75–90 min
Employee Claims practical lab	60–90 min
Knowledge check and research notes	30 min
Total suggested time	4.5–5.75 hours

Official References

• Python Tutorial — Errors and Exceptions: https://docs.python.org/3/tutorial/errors.html

• Python Tutorial — Classes: https://docs.python.org/3/tutorial/classes.html

• Built-in Exceptions: https://docs.python.org/3/library/exceptions.html

• Context Managers: https://docs.python.org/3/library/stdtypes.html#typecontextmanager

• Iterator Types: https://docs.python.org/3/library/stdtypes.html#iterator-types

Study principle: understand the failure, state, and flow of execution before trying to fix the code.